

Resutura: Effect-Semantic Recovery for Computer-Use Agents

Aura Yavary

September 2026

Abstract

Computer-use agents can recover from a failed local action while still leaving the external world in the wrong state. A lost acknowledgement may hide a successful commit; a misdirected write may require a compensating action; another actor may concurrently make a valid change that recovery must preserve; and an irreversible bad effect may admit no safe automated repair. Recent work already covers causal repair, checkpoint rewind, transaction-like tool runtimes, compensation, verify-before-retry, exactly-once execution, irreversible-effect safety invariants, and human-guided harm recovery. We therefore study a narrower question: whether one computer-use recovery runtime can select the correct *effect semantics* after divergence—verify and continue, compensate, preserve concurrent state, or stop—while retaining valid task progress. Resutura records typed postconditions and effect ownership, separates checkpointed local state from committed external effects, and emits auditable recovery certificates. A deterministic diagnostic suite evaluates Base, blind Retry, a Verification-Aware prior-work baseline, Rewind, and Resutura across compensable misdirection, ambiguous commit, concurrent valid change, and non-compensatable misdirection. A second Tool Contract Matrix factors authoritative readback and idempotency under an unobservable ambiguous commit. The results validate runtime semantics only: Verification-Aware matches Resutura when readback alone resolves ambiguity; idempotency lets simple Retry and Rewind avoid duplicates; Resutura adds distinct behavior for ownership-sensitive compensation and non-compensatable outcomes. We additionally validate mechanism transfer in a real headless Chromium DOM, while keeping the action policy deterministic. We do not claim model-backed or state-of-the-art performance; the next empirical gate is a factorized evaluation over model, harness, and tool-contract semantics.

1 Motivation

Long-horizon agents act through interfaces whose effects do not share one rollback domain. A VM snapshot may restore a tab, file, or agent context, while a message, remote record, publication, or payment can remain committed. Recovery therefore cannot be reduced to a single operation such as retry or rewind.

We call the missing decision variable *effect semantics*: what evidence exists about the realized action, whether the effect is already committed, whether replay is idempotent, whether a compensator exists, whether concurrent state belongs to another actor, and whether automation should stop.

The research question is:

Can a computer-use recovery runtime choose the appropriate recovery semantics from realized effect state while preserving already-valid progress?

2 Positioning against 2026 work

The surrounding area is crowded. CausalFlow studies causal attribution and minimal counterfactual repair [1]; CUADebug studies CUA diagnosis and corrective re-execution [2]; AgentRewind and Rollback-Induced Reflection study recovery around checkpoints and rollback boundaries [3, 4]. Robust Agent Compensation, Atomix, and Cordon already cover compensation and transaction-like execution semantics [5, 6, 7]. ACRFence studies irreversible effects across restore [8]. Verification-aware tool wrappers study non-atomic failures and verify-before-retry [9]. Revisable by Design formalizes idempotent, reversible, compensable, and irreversible actions [10]. Recent exactly-once work further shows that model behavior, harness behavior, and

tool contracts all affect duplicate side effects [11]. Safety-invariant work formalizes execution fidelity for irreversible state transitions [12], while Human-Guided Harm Recovery and BackBench study preference-aligned recovery after harmful CUA states [13]. CONTINUITY studies security-context contracts and consequence integrity across composed agent controls [14]; StateAct shows that program-state grounding plus independent finish verification can improve long-horizon computer use [15].

Accordingly, Resutura does *not* claim to invent rollback, compensation, idempotency, causal repair, effect ledgers, or reversibility taxonomies. Its contribution hypothesis is the integration of these concerns into a CUA recovery decision contract and a diagnostic suite where the correct response changes qualitatively across effect states.

3 Effect-semantic recovery contract

Let the joint state be $s = (s^L, s^E)$, where s^L is local/rewindable state and s^E is externally committed state. Each consequential action declares expected postconditions P , an intended target, and an effect contract when available. The runtime also maintains an effect ledger with ownership and compensation status.

After divergence, the controller classifies one of the following cases:

1. **Verified commit.** The action result is ambiguous, but authoritative postconditions show the intended effect happened. Continue without replay.
2. **Compensable wrong effect.** An agent-owned incorrect effect committed and a compensator exists. Apply forward compensation, then the intended effect.
3. **Concurrent valid state.** External state not owned by the failed action is treated as a protected invariant and must survive recovery.
4. **Non-compensatable effect.** The bad effect cannot be safely undone. Stop and escalate rather than report fabricated recovery.
5. **Local reversible failure.** Ordinary local repair, subgoal reconstruction, or rewind remains admissible.

Candidate recovery r can be scored as

$$U(r) = P(\text{recover} \mid r) - \lambda_c C(r) - \lambda_l L(r) - \lambda_s R(r) - \lambda_p \Delta_{\text{progress}}(r) - \lambda_d D_{\text{collateral}}(r),$$

subject to task postconditions and protected invariants. In a forkable deterministic environment, Resutura can execute candidate repairs on cloned failed state for post-hoc analysis. Those observed counterfactual outcomes are explicitly not used for online policy selection; the online controller consumes observable state plus declared tool capabilities only.

4 Prototype

The bundled research implementation uses a deterministic multi-app workflow: gather research facts, populate a spreadsheet, write a comparison document, save artifacts, and publish the result. The environment exposes local application state and a separate external-effect ledger. Checkpoint restore deliberately preserves committed external effects.

The recovery certificate records the diagnosed slice, considered alternatives, postcondition verification, protected invariants, progress preservation, external-effect reconciliation, and safe-abort status.

5 Effect Semantics Suite

We evaluate five deterministic scaffolds—Base, one-shot blind Retry, a Verification-Aware baseline, checkpoint Rewind, and Resutura—with 40 trials per method per scenario.

Scenario	Base	Retry	Verify	Rewind	Resutura
Compensable misdirection	0%	0%	0%	0%	100%
Ambiguous commit	100%	0%	100%	0%	100%
Concurrent valid change + misdirection	0%	0%	0%	0%	100%
Non-compensatable misdirection	0%	0%	0%	0%	0%

Table 1: Task success in the deterministic diagnostic suite. For the final row, Resutura safe-aborts in 100% of trials; zero task success is intentional. These are not real-browser/model results.

Compensable misdirection. The final publish commits to a wrong target. Base, Retry, and Rewind leave the external error unreconciled. Resutura retracts the agent-owned wrong publication and publishes to the intended target.

Ambiguous commit. The intended publish commits, but the caller receives a timeout. Base succeeds by luck because it does not retry. Retry and Rewind each duplicate the intended effect in every trial under the non-idempotent default contract. The Verification-Aware baseline and Resutura both use authoritative readback and continue without replay, as expected from prior work.

Concurrent valid change. Before the misdirected publish, another actor creates a valid external update. An earlier implementation bug naively selected the first non-target publication for compensation, which could delete the other actor’s update. The corrected controller uses effect ownership and preserves the concurrent publication in every trial.

Non-compensatable effect. The wrong effect commits with no safe compensator. Resutura reports zero task success and safe-aborts in every trial rather than presenting an impossible recovery as success.

6 Component ablations

To test whether the integrated behavior is attributable to specific mechanisms rather than only the full controller, we run three component ablations with 40 paired trials per variant and scenario. Removing forward compensation drops task success from 100% to 0% on both compensable wrong-target scenarios. Ignoring tool-contract evidence causes ambiguous commits to be blindly replayed, yielding duplicate intended effects in every trial and 0% task success. Removing effect ownership leaves simple misdirection recoverable but drops the concurrent-valid-change scenario to 0%, because target-only reconciliation cannot safely isolate agent-owned damage from protected external state. The full controller safe-aborts all non-compensatable cases rather than optimizing for task completion at the expense of false recovery. These ablations establish mechanism necessity only inside the deterministic testbed.

Variant	Misdirect	Ambiguous	Concurrent	Non-compensatable
Full Resutura	100%	100%	100%	100% abort
– compensation	0%	100%	0%	100% abort
– contract awareness	100%	0%	100%	100% abort
– effect ownership	100%	100%	0%	100% abort

Table 2: Deterministic component ablations, 40 paired task/fault seeds per cell.

7 Dataset and uncertainty reporting

The release packages 160 public scenario/task records and 800 raw method trajectories as JSONL/GZIP, together with a schema and source-run hash. The public dev/test labels are for analysis hygiene only and are

not hidden evaluation data. For the deterministic simulator we report Wilson 95% binomial intervals and paired-seed contrasts as descriptive diagnostics. They must not be interpreted as uncertainty over frontier-model or production performance. The Chromium mechanism-transfer tier has only two paired trials per method/scenario and is intentionally not used for statistical or SOTA claims.

8 Tool Contract Matrix

The first suite uses a contract with authoritative readback and no idempotency. To expose dependence on the tool rather than the runtime alone, we add an unobservable ambiguous-commit fault: the write commits, but the failed call result contains no evidence of the commit. We then factor readback and idempotency, with 40 trials per method per contract.

Contract	Base	Retry	Verify	Rewind	Resutura
Readback only	100%	0%	100%	0%	100%
Idempotency only	100%	100%	100%	100%	100%
Readback + idempotency	100%	100%	100%	100%	100%
Neither	100%*	0%	0% [†]	0%	0% [†]

Table 3: Ground-truth task success under an unobservable ambiguous commit. *Base leaves the world correct because it does not replay, but it has no evidence that the commit occurred. [†]Verification-Aware and Resutura safe-abort in 100% of trials.

The matrix deliberately removes any universal “Resutura wins” story. Authoritative readback is sufficient for the Verification-Aware baseline to match Resutura on this fault. Tool-level idempotency is sufficient for Retry and Rewind to avoid duplicates. With neither capability, a runtime cannot generally infer whether replay is safe from the failed acknowledgement alone; conservative abort avoids duplication but sacrifices known completion. This factorization is consistent with recent evidence that exactly-once behavior depends strongly on tool contracts [9, 11].

The matrix also exposed a simulator bug: checkpoint restore originally rewound the server-side idempotency registry. That behavior was incorrect because deduplication state belongs to the external rollback domain. The registry now survives local rewind.

9 Policy boundary and Chromium mechanism transfer

To avoid conflating recovery logic with a particular model vendor, the release exposes a provider-neutral policy contract. A policy receives task context, the current observation, declared tool capabilities, and recent history, and returns the next action plus expected postconditions. The recovery runtime independently executes, verifies, classifies divergence, and selects recovery semantics. The bundled recorded-policy experiment validates this boundary on 20 faulted trials but is explicitly not model evidence.

We also port the workflow to a real headless Chromium DOM. In a paired mechanism-transfer diagnostic (two deterministic trials per method and scenario), the qualitative simulator findings survive: Resutura reconciles wrong-target committed effects, preserves a concurrent valid publication while compensating its own misdirection, safe-aborts on a non-compensatable committed error, and matches verification-aware recovery on an ambiguous commit. The Chromium tier is evidence that the mechanism is not an artifact of Python dictionaries; because the action policy remains deterministic and the external effect ledger is controlled by the harness, it is not evidence about frontier-model computer-use ability.

10 Why task success is insufficient

The ambiguous-commit case shows why aggregate completion can hide bad recovery semantics. Base, Verification-Aware, and Resutura can all score 100% task success on ambiguous commit, but for different reasons: Base performs no recovery, while the latter two explicitly resolve the ambiguity from the declared contract. Conversely, retry-oriented systems can reduce task success by duplicating an already-committed effect. Useful evaluation therefore needs effect-level measures such as duplicate-effect rate, unreconciled-effect rate, protected-state loss, collateral damage, and safe-abort calibration.

11 Real-world falsification study

The next experiment should not hand effect semantics to a toy controller. It should factor three sources of reliability:

1. **Model:** can the model infer the realized state and choose an appropriate action?
2. **Harness:** does runtime logic enforce verify, preserve, compensate, or stop behavior?
3. **Tool contract:** are status/read-back, idempotency keys, compensation, authorization, and commit semantics available?

A side-effect-capable browser/desktop benchmark should include lost acknowledgements, delayed visibility, redelivery, compensable and non-compensatable writes, concurrent legitimate updates, and authorization boundaries. Baselines should include retry, RCA+re-execution, rewind/reflection, verification-aware wrappers, and transactional/compensation runtimes where reproducible.

The hypothesis should be considered unsupported if the Verification-Aware baseline matches Resutura across the broader effect classes (not merely ambiguous commit), if transactional staging dominates without sacrificing completion, if the effect classifier requires privileged labels unavailable to real agents, or if safe abort is poorly calibrated.

12 Limitations

The primary environment is deterministic; an additional headless-Chromium tier exercises real DOM state while keeping the planner deterministic. Effect contracts and compensators are explicit; the failure localizer is rule-based; no frontier model drives the actions; and consequential external services remain harness-controlled. The suite establishes that the recovery semantics are implemented and behaviorally distinguishable, not that a model can infer them in open-world computer use. It also does not establish novelty over every contemporaneous systems paper or SOTA performance.

13 Conclusion

A failed tool result does not uniquely determine a recovery action. Depending on realized effect state, the correct response may be to continue, compensate, preserve concurrent state, rewind, or stop. Resutura turns that distinction into an executable CUA recovery contract and a falsifiable diagnostic suite. The remaining question—and the one that matters for a full paper—is whether this contract improves reliability when effect semantics must be inferred under real computer-use uncertainty.

References

- [1] A. Bonagiri et al. *CausalFlow: Causal Attribution and Counterfactual Repair for LLM Agent Failures*. arXiv:2605.25338, 2026.
- [2] W. Zhang et al. *CUADebug: Diagnosing and Repairing Computer-Use Agent Failures*. arXiv:2608.02643, 2026.

- [3] Y. Zhuang et al. *AgentRewind: Recoverable Execution for Long-Horizon LLM Agents*. arXiv:2608.14380, 2026.
- [4] Y. Yu, L. Yao, Y. Li, E. Wang, and L. Wu. *Rollback the World, Keep the Reflection: Rollback-Induced Reflection for Long-Horizon LLM Agents*. arXiv:2609.18304, 2026.
- [5] S. Perera, K. Hapuarachchi, F. Leymann, and R. Khalaf. *Robust Agent Compensation (RAC): Teaching AI Agents to Compensate*. arXiv:2605.03409, 2026.
- [6] B. Mohammadi et al. *Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows*. arXiv:2602.14849, 2026.
- [7] Z. Chen et al. *Cordon: Semantic Transactions for Tool-Using LLM Agents*. arXiv:2606.17573, 2026.
- [8] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn. *ACRFence: Preventing Semantic Rollback Attacks in Agent Checkpoint-Restore*. arXiv:2603.20625, 2026.
- [9] I. K. Mansoor, A. Phadke, and P. Rana. *Verified Tool Calls Improve LLM Agent Reliability Under Non-Atomic Failures*. arXiv:2608.02645, 2026.
- [10] Z. Zhai, M. Li, and X. Wang. *Revisable by Design: A Theory of Streaming LLM Agent Execution*. arXiv:2604.23283, 2026.
- [11] J. Li. *Where Does Exactly-Once Live? Model, Harness, and Tool-Contract Effects on Duplicate Side Effects in LLM Agents*. arXiv:2609.29095, 2026.
- [12] Z. Yin. *Safety Invariants for Agents Orchestrating Irreversible State Transitions: A Four-Dimensional Formalism Evaluated on Public Ledgers*. arXiv:2608.00783, 2026.
- [13] C. Li, S. CH-Wang, A. Peng, and A. Bobu. *Human-Guided Harm Recovery for Computer Use Agents*. arXiv:2604.18847, 2026.
- [14] C. Zheng and G. Yang. *CONTINUITY: Security-Context Contracts for Composable LLM Agent Controls*. arXiv:2609.05269, 2026.
- [15] Y. Yang et al. *StateAct: Program State, before Pixels, for Long-Horizon Computer-Use Agents*. arXiv:2607.22798, 2026.